

Machine Learning, Assignment 1

Date posted: Oct. 31, 2011,

Due: Nov. 14, 2011 at the *start* of class

1. (4 points) Show that the maximum likelihood solution for a Bernoulli distribution, given training observations $x_1, \dots, x_N$ is $\frac{1}{N} \sum_n x_n$ by setting the derivative of the log-likelihood to zero. Discuss how one may use this fact to derive the maximum-likelihood solution of a Bernoulli Naive Bayes classifier.
2. (4 points) In class, we discussed Gaussian generative models for classification, and discrete Naive Bayes classifiers. Discuss how one may define a *Gaussian naive Bayes classifier*. Describe what its parameters are, and write down their maximum likelihood estimates.
3. (2 points) This question is to get you started with some simple data analysis in Python. Get access to a Python environment containing the packages numpy and matplotlib. Download the files

`http://www.ml.cs.uni-frankfurt.de/teaching/ml2011/assignments/housing.data`

and

`http://www.ml.cs.uni-frankfurt.de/teaching/ml2011/assignments/housing\_descr.txt`

and save them locally under the same names. Take a look at the contents of the files.

Then run the following code:

```
import numpy
import pylab

data = numpy.loadtxt("housing.data")
inputs = data[:, :-1]
outputs = data[:, -1]
N, D = inputs.shape

pylab.subplot(1, 2, 1)
pylab.scatter(inputs[:,0], outputs)
pylab.xlabel("crime rate")
pylab.ylabel("median neighborhood house price in 1000USD")
pylab.subplot(1, 2, 2)
pylab.scatter(inputs[:,5], outputs)
pylab.xlabel("average number of rooms per dwelling")
pylab.ylabel("median neighborhood house price in 1000USD")
pylab.show()
```

Explain briefly what you see, and why.

What to hand in: A print-out of the plots, and your explanation. Do not hand in any code.

4. (5 points) Preprocess the data-set from the previous question by mean-centering each dimension, followed by normalizing the variance in each dimension to be 1, followed by adding a constant 1-dimension. You may use the following Python-code for this purpose:

```
#canonical pre-processing
inputs -= inputs.mean(0)[numpy.newaxis, :]
inputs /= inputs.std(0)[numpy.newaxis, :]
inputs = numpy.concatenate((inputs, numpy.ones((N, 1))), 1)
```

Estimate a **linear regression** model to predict t (given by variable “outputs” in the previous question) from the normalized inputs $\mathbf{x}$ (given by variable “inputs”), using the closed-form solution we discussed in class. The linear regression model is defined by the 14-dimensional weight vector $\mathbf{w}$ which minimizes $E(\mathbf{w}) = \sum_n (t_n - \mathbf{w}^T \mathbf{x}_n)^2$.

Make a plot of the 14 regression coefficients. Then train a ridge regression model which minimizes $E(\mathbf{w}) + 100.0 \|\mathbf{w}\|^2$. Add a plot of the learned coefficients in the same plot using a different color and/or a different line-style.

What is the total squared reconstruction error $E(\mathbf{w}) = \sum_n (t_n - \mathbf{w}^T \mathbf{x}_n)^2$ on the training data, for each model?

Hints:

- To invert a matrix you can use the function `numpy.linalg.inv()`

What to hand in: A print-out of your plot, the two reconstruction costs. Do not hand in any code.

5. (5 points) Train the linear regression coefficients again, this time using **stochastic gradient descent**, as follows: Initialize $\mathbf{w}$ to small random values. Then repeat the following 1000 times: {For each training case, update $\mathbf{w}$ by subtracting η times the gradient $\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}}$ of the squared error $E(\mathbf{w})$ with respect to this case, then compute and save off the current *total* squared error computed on the whole training set, using the just updated $\mathbf{w}$.} Experiment to find a value for η that you think is reasonable.

What to hand in: The value of η that you chose; a plot of the training error as a function of the iteration count (1000 values). Do not hand in any code.